Name : Vaishnavi Eknath Avhad
Class : D15C
Roll No. : 14

# Practical 3

| Aim : Apply Decision Tree and Random Forest for classification tasks |
|---|

**1. Dataset Source**

The dataset used for this experiment is the Breast Cancer Wisconsin (Diagnostic) Dataset.

- Source Link : Kaggle - Breast Cancer Wisconsin (Diagnostic) Data

---

**2. Dataset Description**

This dataset consists of 569 instances with 30 numeric predictive features computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. It is used to classify tumors as either Malignant or Benign.

| Feature | Description | Type |
|---|---|---|
| diagnosis | The target variable (M = malignant, B = benign). | Categorical (Target) |
| radius_mean | Mean of distances from center to points on the perimeter. | Numeric |
| texture_mean | Standard deviation of gray-scale values. | Numeric |
| area_mean | Mean area of the tumor cell. | Numeric |
| smoothness_mean | Mean of local variation in radius lengths. | Numeric |
| concavity_mean | Mean of severity of concave portions of the contour. | Numeric |

---

**3. Mathematical Formulation**

### A. Decision Tree Classifier

A Decision Tree is a non-parametric supervised learning method. It utilizes a tree-like graph of decisions. The tree is built by splitting the source set into subsets based on an attribute value test, using metrics like Gini Impurity or Information Gain.

Gini Impurity Formula :

$$Gini(p) = 1 - \sum_{i=1}^{J} p_i^2$$

Where $p_i$ is the probability of an object being classified into a particular class.

### B. Random Forest Classifier

Random Forest is an ensemble learning method that operates by constructing a multitude of decision trees at training time. For classification tasks, the output is the class selected by most trees (majority voting).

Key Principles :

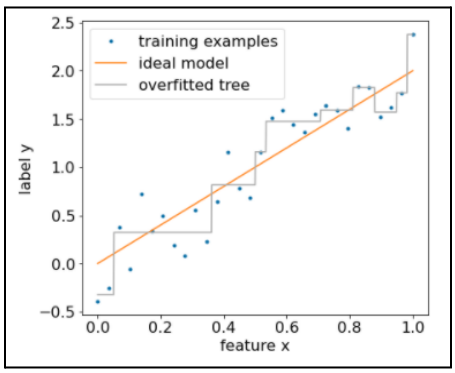
- Bagging (Bootstrap Aggregating) : Training each tree on a random subset of the data.
- Feature Randomness : Selecting a random subset of features at each split to ensure trees are de-correlated.

---

## 4. Algorithm Limitations

### A. Decision Tree Limitations

1. Overfitting : Trees can easily become over-complex, capturing noise in the training data rather than the underlying pattern.

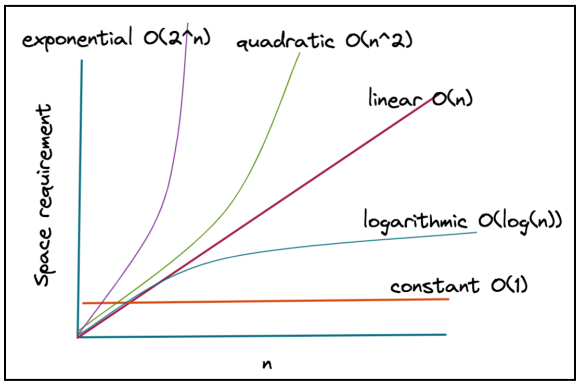


2. Instability : Small variations in the data might result in a completely different tree being generated.

3. Bias : Decision trees can be biased if some classes dominate.

**B. Random Forest Limitations**

1. Complexity : Random forests are much more computationally expensive and slower than individual decision trees.

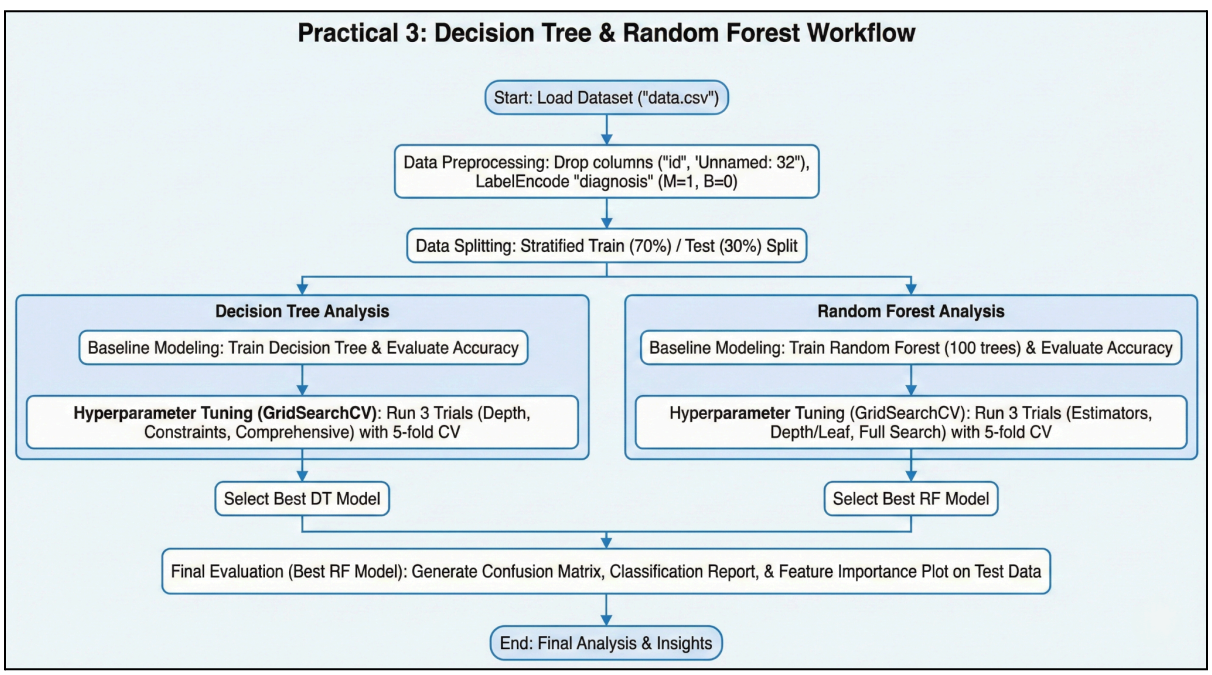



2. Lack of Interpretability : While a single tree is easy to visualize, a forest of 100+ trees acts as a "black box."

3. Memory Usage : Storing many trees can require significant memory for large datasets.

---

## 5. Methodology / Workflow



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
df = pd.read_csv('data.csv')
target_column = 'diagnosis'
df = df.drop(['id', 'Unnamed: 32'], axis=1)
le = LabelEncoder()
df[target_column] = le.fit_transform(df[target_column])
X = df.drop(target_column, axis=1)
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
print("\n" + "="*20 + " DECISION TREE ANALYSIS " + "="*20)
# Baseline Model
dt_baseline = DecisionTreeClassifier(random_state=42)
dt_baseline.fit(X_train, y_train)
y_pred_dt = dt_baseline.predict(X_test)
print(f"Baseline Accuracy: {accuracy_score(y_test, y_pred_dt) * 100:.2f}%")
# Decision Tree Tuning Trials
dt_trials = {
    "Trial 1 (Depth Control)": {'max_depth': [3, 5, 7]},
     "Trial 2 (Leaf/Split Constraints)": {'min_samples_split': [2, 5, 10], 'min_samples_leaf': [1, 2,
4]},
     "Trial 3 (Comprehensive Search)": {'max_depth': [None, 5, 10], 'min_samples_split': [2, 5],
'min_samples_leaf': [1, 2]}
}
dt_results = []
for name, params in dt_trials.items():
            grid = GridSearchCV(DecisionTreeClassifier(random_state=42), params, cv=5,
scoring='accuracy')
    grid.fit(X_train, y_train)
    best_model = grid.best_estimator_
    acc = accuracy_score(y_test, best_model.predict(X_test))
    dt_results.append({
        "Trial": name,
        "Best Params": grid.best_params_,
        "Test Accuracy": acc
    })
# Display DT Results
```

```
dt_results_df = pd.DataFrame(dt_results)
print("\nDecision Tree Hyperparameter Tuning Summary:")
print(dt_results_df.to_string(index=False))
print("\n" + "="*20 + " RANDOM FOREST ANALYSIS " + "="*20)
# Baseline Model
rf_baseline = RandomForestClassifier(n_estimators=100, random_state=42)
rf_baseline.fit(X_train, y_train)
y_pred_rf = rf_baseline.predict(X_test)
print(f"Baseline Accuracy: {accuracy_score(y_test, y_pred_rf) * 100:.2f}%")
# Random Forest Tuning Trials
rf_trials = {
    "Trial 1 (Estimator Count)": {'n_estimators': [50, 100, 200]},
    "Trial 2 (Depth & Leaf Constraints)": {'max_depth': [5, 10], 'min_samples_leaf': [2, 4]},
        "Trial 3 (Full Parameter Search)": {'n_estimators': [100, 150], 'max_depth': [None, 10],
'min_samples_split': [2, 5]}
}
rf_results = []
for name, params in rf_trials.items():
            grid = GridSearchCV(RandomForestClassifier(random_state=42), params, cv=5,
scoring='accuracy')
    grid.fit(X_train, y_train)
    best_model = grid.best_estimator_
    acc = accuracy_score(y_test, best_model.predict(X_test))
    rf_results.append({
        "Trial": name,
        "Best Params": grid.best_params_,
        "Test Accuracy": acc
    })
# Display RF Results
rf_results_df = pd.DataFrame(rf_results)
print("\nRandom Forest Hyperparameter Tuning Summary:")
print(rf_results_df.to_string(index=False))
final_rf_model = grid.best_estimator_
y_pred_final = final_rf_model.predict(X_test)
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred_final), annot=True, fmt='d', cmap='Greens')
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Best Random Forest")
plt.show()
print("\nFinal Classification Report (Best Random Forest):")
print(classification_report(y_test, y_pred_final))
importances = final_rf_model.feature_importances_
indices = np.argsort(importances)[::-1]
plt.figure(figsize=(10, 6))
plt.title("Feature Importances - Random Forest")
```

```
plt.bar(range(X.shape[1]), importances[indices], align='center', color='forestgreen')
plt.xticks(range(X.shape[1]), X.columns[indices], rotation=90)
plt.xlabel("Features")
plt.ylabel("Importance Score")
plt.tight_layout()
plt.show()
```

---

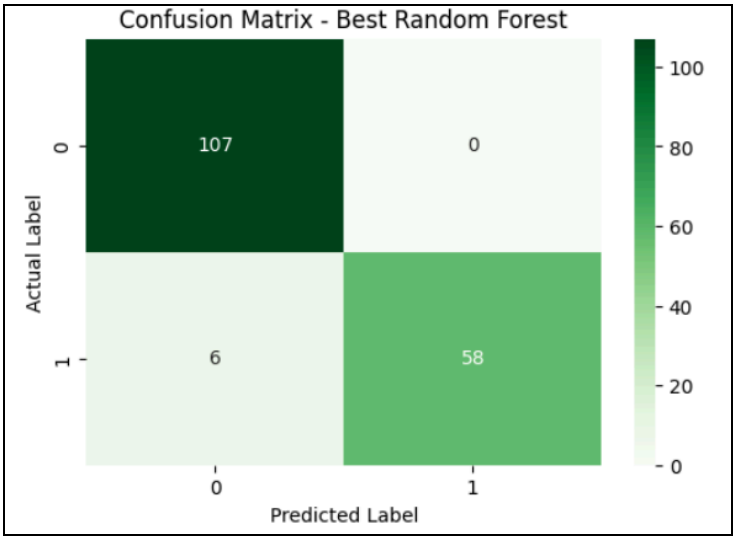
**6. Performance Analysis**

**A. Baseline Results**

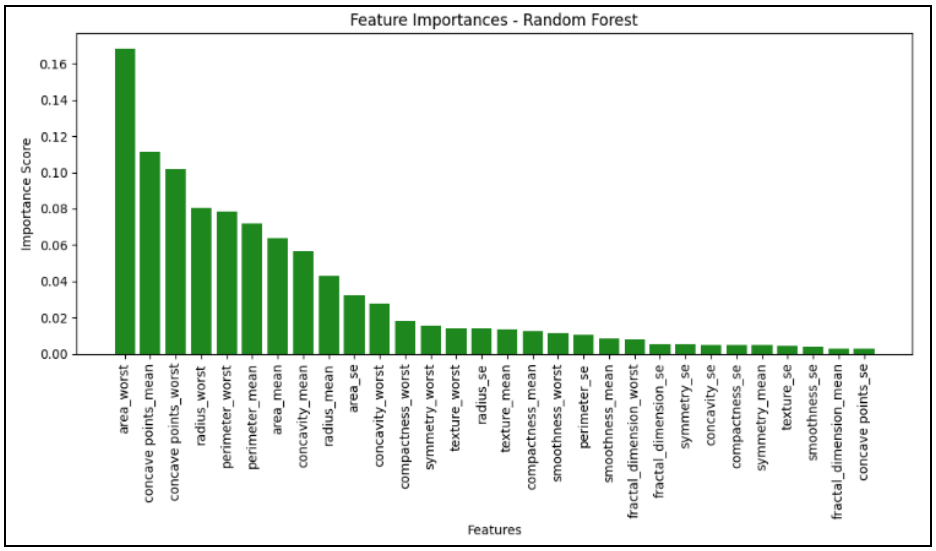
| Model | Baseline Accuracy |
|---|---|
| Decision Tree | 90.06% |
| Random Forest | 96.49% |

**B. Visual Analysis**

- Confusion Matrix : Used to identify True Positives (correctly identified Malignant) and True Negatives (correctly identified Benign).




- Feature Importance : This plot ranks biological markers (like concave points_mean) by their predictive power, showing which factors most influence a cancer diagnosis.




---

### 7. Hyperparameter Tuning (Experimental Trials)

We conducted three trials for both models to observe how constraints on tree growth and forest size affect accuracy.

**Decision Tree Trials**

| Trial | Focus | Best Params | Test Accuracy |
|---|---|---|---|
| Trial 1 | Depth Control | max_depth | 0.906433 |
| Trial 2 | Leaf/Split Constraints | min_samples_leaf | 0.929825 |
| Trial 3 | Comprehensive Search | Mixed | 0.929825 |

**Random Forest Trials**

| Trial | Focus | Best Params | Test Accuracy |
|---|---|---|---|
| Trial 1 | Estimator Count | n_estimators | 0.964912 |
| Trial 2 | Depth & Leaf Constraints | max_depth | 0.959064 |
| Trial 3 | Full Parameter Search | Mixed | 0.964912 |

Tuning Interpretation :

- Regularization : In Decision Trees, Trial 1 (Depth Control) prevents the tree from growing too deep, which reduces overfitting.
- Ensemble Power : In Random Forest, Trial 1 shows if increasing the number of trees improves stability or leads to diminishing returns.

---

**8. Conclusion**

In this practical, Decision Tree and Random Forest models were successfully applied to the Breast Cancer Diagnostic dataset. While the Decision Tree provided a clear, interpretable model, it was more prone to variance. The Random Forest ensemble significantly improved accuracy and robustness. The hyperparameter tuning trials demonstrated that controlling tree depth and the number of estimators is critical for achieving a high-performing, generalized classification model.